



TECNOLÓGICO DE ESTUDIOS SUPERIORES DE VILLA GUERRERO

INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA

SISTEMAS OPERATIVOS PRACTICA 1

PROFESOR

ISC. REYNOSO ELIAS JUAN

ALUMNO

JOSE ANGEL GARCIA CABRERA

GRUPO

3501



Contenido

Introducción	3
1.PS	3
2.PS -EF	3
3.PS –EF GREP FIREFOX	4
4.PS –EO PID,COMM	5
Conclusión.....	5

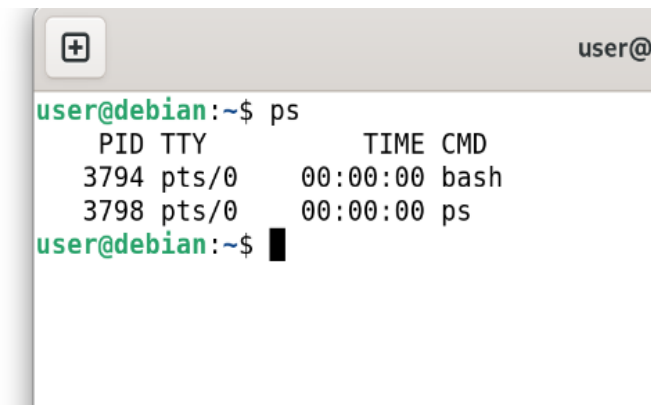


Introducción

En la administración de sistemas operativos basados en Linux, el monitoreo y control de los procesos en ejecución constituye una tarea fundamental para garantizar el uso eficiente de los recursos del sistema y mantener un rendimiento óptimo. En este contexto, el entorno de línea de comandos en distribuciones como Debian ofrece herramientas esenciales para examinar las tareas en segundo y primer plano. La presente práctica se enfoca en la exploración y uso del comando `ps`, analizando desde sus parámetros de búsqueda básicos hasta sus opciones de formateo avanzado y filtrado mediante tuberías, con la finalidad de comprender la estructura, jerarquía y estado de los procesos dentro del sistema operativo.

1.PS

En esta primera captura se realiza la ejecución básica del comando `ps`, el cual sirve para consultar únicamente los procesos que pertenecen al usuario que inició la sesión y que se están ejecutando dentro de la terminal interactiva actual. En el resultado generado se observa una estructura sencilla donde destacan la terminal `bash` que procesa la entrada y el mismo comando `ps` que fue invocado para realizar la consulta.



```
user@debian:~$ ps
  PID TTY          TIME CMD
 3794 pts/0        00:00:00 bash
 3798 pts/0        00:00:00 ps
user@debian:~$
```

2.PS -EF

En la segunda captura se lleva a cabo la ejecución del comando `ps -ef`, en donde la opción `-e` le indica al sistema que debe mostrar todos los procesos que están corriendo en la máquina y la opción `-f` solicita un formato de salida detallado. En la pantalla se despliega un listado completo con columnas explicativas como el usuario propietario, el identificador único del proceso, el identificador del proceso padre que le dio origen, la hora exacta de inicio y el comando o hilo del núcleo del sistema operativo en ejecución.



```
user@debian:~$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root         1      0  0 07:35 ?        00:00:02 /sbin/init components splash
root         2      0  0 07:35 ?        00:00:00 [kthreadd]
root         3      2  0 07:35 ?        00:00:00 [pool_workqueue_release]
root         4      2  0 07:35 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
root         5      2  0 07:35 ?        00:00:00 [kworker/R-rcu_gp]
root         6      2  0 07:35 ?        00:00:00 [kworker/R-sync_wq]
root         7      2  0 07:35 ?        00:00:00 [kworker/R-slub_flushwq]
root         8      2  0 07:35 ?        00:00:00 [kworker/R-netns]
root         9      2  0 07:35 ?        00:00:00 [kworker/0:0-events]
root        11      2  0 07:35 ?        00:00:00 [kworker/0:0H-events_highpri]
root        12      2  0 07:35 ?        00:00:00 [kworker/u32:0-events_unbound]
root        13      2  0 07:35 ?        00:00:00 [kworker/R-mm_percpu_wq]
root        14      2  0 07:35 ?        00:00:00 [rcu_tasks_kthread]
root        15      2  0 07:35 ?        00:00:00 [rcu_tasks_rude_kthread]
root        16      2  0 07:35 ?        00:00:00 [rcu_tasks_trace_kthread]
root        17      2  0 07:35 ?        00:00:00 [ksoftirqd/0]
root        18      2  0 07:35 ?        00:00:00 [rcu_preempt]
root        19      2  0 07:35 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/0]
root        20      2  0 07:35 ?        00:00:00 [rcu_exp_gp_kthread_worker]
root        21      2  0 07:35 ?        00:00:00 [migration/0]
root        22      2  0 07:35 ?        00:00:00 [idle_inject/0]
root        23      2  0 07:35 ?        00:00:00 [cpuhp/0]
root        24      2  0 07:35 ?        00:00:00 [cpuhp/2]
root        25      2  0 07:35 ?        00:00:00 [idle_inject/2]
root        26      2  0 07:35 ?        00:00:00 [migration/2]
root        27      2  0 07:35 ?        00:00:00 [ksoftirqd/2]
root        28      2  0 07:35 ?        00:00:00 [kworker/2:0-events]
root        29      2  0 07:35 ?        00:00:00 [kworker/2:0H-events_highpri]
root        30      2  0 07:35 ?        00:00:00 [cpuhp/4]
root        31      2  0 07:35 ?        00:00:00 [idle_inject/4]
root        32      2  0 07:35 ?        00:00:00 [migration/4]
```

3.PS -EF | GREP FIREFOX

En la tercera captura se aplica el uso de una tubería mediante la instrucción `ps -ef | grep firefox` con el objetivo de filtrar el listado general de procesos e identificar si existe alguno relacionado con el programa Firefox. Como la salida no muestra ninguna instancia del navegador y únicamente devuelve la línea correspondiente a la propia búsqueda realizada por `grep`, se entiende que el navegador no se encontraba en ejecución durante esa consulta.

```
user@debian:~$ ps -ef | grep firefox
user          4060    4024  0 07:45 pts/0    00:00:00 grep firefox
user@debian:~$
```



4.PS –EO PID,COMM

En la cuarta captura se observa el uso del comando `ps -eo pid,comm`, mediante el cual se le aplica un formato personalizado a la salida para mostrar únicamente las columnas de interés. Gracias a la opción `-o`, la consulta omite los campos adicionales y presenta un reporte limpio y simplificado que contiene exclusivamente la columna del identificador de proceso junto con el nombre corto de la orden o servicio asociado.

```
user@debian:~$ ps -eo pid,comm
PID COMMAND
1 systemd
2 kthreadd
3 pool_workqueue_release
4 kworker/R-kvfree_rcu_reclaim
5 kworker/R-rcu_gp
6 kworker/R-sync_wq
7 kworker/R-slub_flushwq
8 kworker/R-netns
9 kworker/0:0-events
11 kworker/0:0H-events_highpri
13 kworker/R-mm_percpu_wq
14 rcu_tasks_kthread
15 rcu_tasks_rude_kthread
16 rcu_tasks_trace_kthread
17 ksoftirqd/0
18 rcu_preempt
19 rcu_exp_par_gp_kthread_worker/0
20 rcu_exp_gp_kthread_worker
21 migration/0
22 idle_inject/0
23 cpuhp/0
24 cpuhp/2
25 idle_inject/2
26 migration/2
27 ksoftirqd/2
28 kworker/2:0-events
29 kworker/2:0H-events_highpri
30 cpuhp/4
31 idle_inject/4
32 migration/4
33 ksoftirqd/4
```

Conclusión

A través del desarrollo de esta práctica se logró dominar el uso de la herramienta `ps` para la inspección y análisis de procesos en un entorno Debian. Se comprendió la diferencia entre visualizar los procesos locales de una terminal y examinar la totalidad de las tareas del sistema, identificando parámetros clave como los identificadores de proceso (`PID`), procesos padre (`PPID`) e hilos del kernel. Asimismo, la integración de la herramienta de filtrado `grep` demostró la efectividad de las tuberías para la localización específica de tareas, mientras que la personalización de columnas confirmó la flexibilidad de la línea de comandos para adaptar los reportes a las necesidades del administrador. Estos conocimientos resultan esenciales para el diagnóstico de fallas, la gestión de memoria y el control operativo de un sistema Linux.